TestNG Complete Revision Guide for Selenium with Java

---

1. Setup TestNG in Maven

```
Unset
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>7.8.0</version>
</dependency>
```

---

2. TestNG Annotations Overview

Suite-Level Annotations:

- `@BeforeSuite`, `@AfterSuite`

Test-Level Annotations:

- `@BeforeTest`, `@AfterTest`

Class-Level Annotations:

- `@BeforeClass`, `@AfterClass`

Method-Level Annotations:

- `@BeforeMethod`, `@AfterMethod`

Test Case Annotation:

- `@Test`

Execution Order: `@BeforeSuite` → `@BeforeTest` → `@BeforeClass` → `@BeforeMethod` → `@Test` → `@AfterMethod` → `@AfterClass` → `@AfterTest` → `@AfterSuite`

---

3. Execution Control

Priority Based Execution:

```
Java
@Test(priority = 1)
public void launchBrowser() {}
```

---

4. Grouping Tests

```
Java
@Test(groups = "smoke")
public void homePageTest() {}
```

testng.xml:

```
Unset
<groups>
  <run>
    <include name="smoke"/>
  </run>
</groups>
```

---

5. Parameters & DataProvider

XML Parameter:

```
Unset
<parameter name="url" value="https://example.com"/>
```

```java
@Test
@Parameters({"url"})
public void test(String url) {}
```

DataProvider:

```java
@DataProvider(name = "loginData")
public Object[][] provideData() {
  return new Object[][] {{"user1", "pass1"}, {"user2", "pass2"}};
}
```

```java
@Test(dataProvider = "loginData")
public void test(String user, String pass) {}
```

External Class DataProvider:

```java
@Test(dataProvider = "loginData", dataProviderClass =
DataProviders.class)
```

---

6. Test Dependencies

```java
@Test(dependsOnMethods = "setup")
public void mainTest() {}
```

---

7. Assertions

```java
assertEquals(actual, expected);
assertTrue(condition);
```

SoftAssert Example:

```java
SoftAssert softAssert = new SoftAssert();
softAssert.assertEquals(1, 2);
softAssert.assertAll();
```

---

8. Parallel Execution

1. Parallel by Methods

```unset
<suite name="ParallelMethodSuite" parallel="methods"
thread-count="3">
  <test name="TestMethods">
    <classes>
      <class name="com.example.ParallelTest"/>
    </classes>
  </test>
</suite>
```

2. Parallel by Classes

```unset
<suite name="ParallelClassSuite" parallel="classes"
thread-count="3">
  <test name="TestClasses">
    <classes>
      <class name="com.example.Class1"/>
      <class name="com.example.Class2"/>
```

```
    </classes>
  </test>
</suite>
```

3. Parallel by Tests

```
Unset
<suite name="ParallelTestsSuite" parallel="tests"
thread-count="2">
  <test name="Test1">
    <classes>
      <class name="com.example.TestA"/>
    </classes>
  </test>
  <test name="Test2">
    <classes>
      <class name="com.example.TestB"/>
    </classes>
  </test>
</suite>
```

4. Parallel by Instances

```
Unset
<suite name="InstanceParallelSuite" parallel="instances"
thread-count="2">
  <test name="InstanceTest">
    <classes>
      <class name="com.example.InstanceTestClass"/>
    </classes>
  </test>
</suite>
```

Use this with @Factory to create multiple instances of the same test class.

---

9. TestNG Listeners

Implementing ITestListener:

```java
public class CustomListener implements ITestListener {
  public void onTestSuccess(ITestResult result) {}
  public void onTestFailure(ITestResult result) {}
  public void onStart(ITestContext context) {}
  public void onFinish(ITestContext context) {}
}
```

testng.xml:

```unset
<listeners>
  <listener class-name="com.example.CustomListener"/>
</listeners>
```

Using @Listeners Annotation:

```java
@Listeners(CustomListener.class)
public class MyTestClass {}
```

---

10. TestNG Reports

- Default HTML reports: `test-output/emailable-report.html`

ExtentReports Listener Example:

- Implements rich reports with screenshots

- Requires `extentreports` dependency

---

11. Expected Exceptions

```java
@Test(expectedExceptions = IllegalArgumentException.class)
public void test() {
  throw new IllegalArgumentException("Expected exception");
}
```

With Regex Message Match:

```java
@Test(expectedExceptionsMessageRegExp = "Expected.*")
```

---

12. Timeouts

```java
@Test(timeOut = 5000)
public void timeoutTest() {}
```

---

13. Skipping Tests

```java
@Test(enabled = false)
public void skippedTest() {}
```

---

14. Factory Pattern

```java
@Factory
public Object[] createTests() {
  return new Object[] { new ParameterizedTest("val1") };
}
```

---

15. Sample testng.xml Structure

```unset
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="SampleSuite">
  <test name="RegressionTests">
    <parameter name="browser" value="chrome"/>
    <groups>
      <run>
        <include name="regression"/>
      </run>
    </groups>
    <classes>
      <class name="com.example.LoginTests"/>
    </classes>
  </test>
</suite>
```

---

16. Running Tests

- **IDE:** Right-click → Run
- **Maven:** `mvn test`
- **CLI:** `java -cp ... org.testng.TestNG testng.xml`

---

17. Configuration Failures

```java
@Test(configFailurePolicy = ConfigurationFailurePolicy.CONTINUE)
```

---

18. Retry Failed Tests (IRetryAnalyzer)

```java
public class RetryAnalyzer implements IRetryAnalyzer {
  private int count = 0;
  private static final int MAX_RETRY = 2;

  @Override
  public boolean retry(ITestResult result) {
    if (count < MAX_RETRY) {
      count++;
      return true;
    }
    return false;
  }
}
```

```java
@Test(retryAnalyzer = RetryAnalyzer.class)
public void flakyTest() {}
```

---

19. Transformers (Optional for Advanced Use)

- Used for dynamically changing test behavior at runtime.
- Interface: `IAnnotationTransformer`

```java
public class AnnotationTransformer implements
IAnnotationTransformer {
  public void transform(ITestAnnotation annotation, Class
testClass, Constructor testConstructor, Method testMethod) {
    annotation.setRetryAnalyzer(RetryAnalyzer.class);
  }
}
```

testng.xml or via @Listeners to activate transformer:

```unset
<listeners>
  <listener class-name="com.example.AnnotationTransformer"/>
</listeners>
```

End of Document